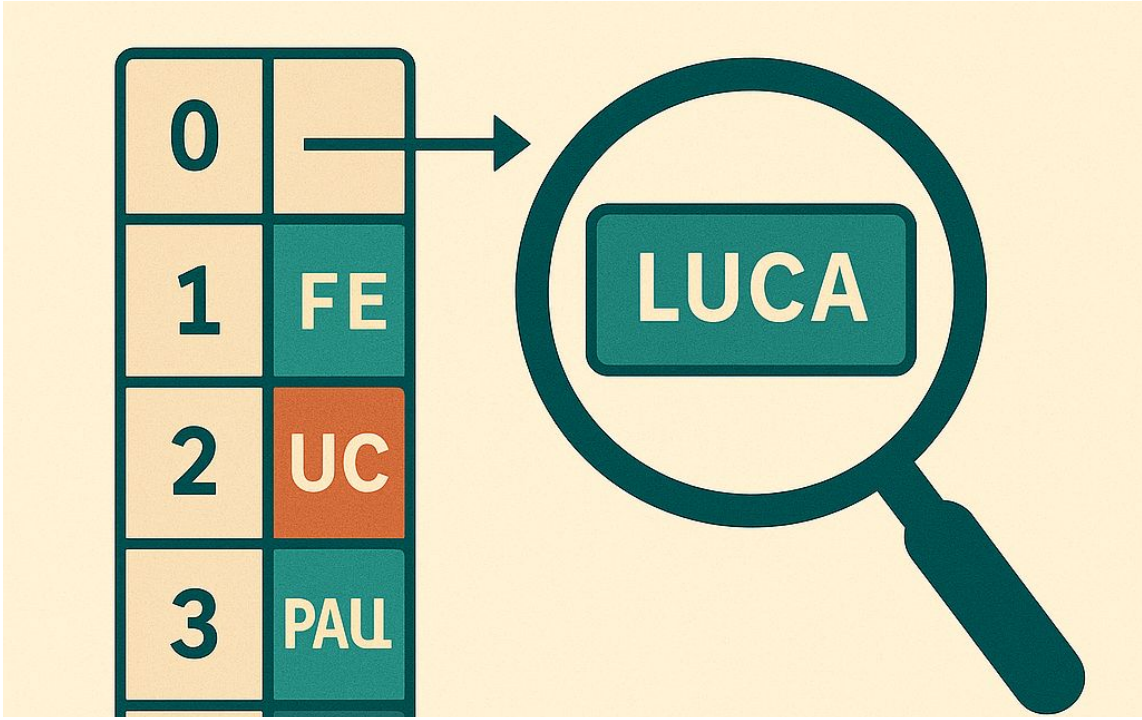
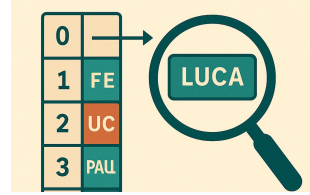
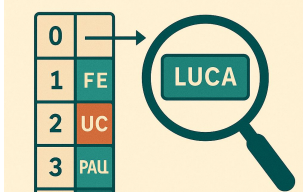
Algoritmos de Busca





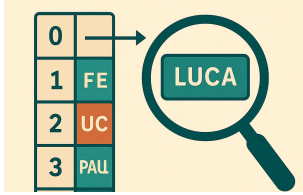
Algoritmos de Busca

- Algoritmos de busca fornecem soluções para o problema computacional de busca.
- Problema Computacional de Busca
  - Input: Uma sequência de n dados $\{a_1\ a_2, a_3, ..., a_n\}$ e dado a_x a ser encontrado.
  - Output: Verdadeiro, se $a_x \in \{a_1\ a_2, a_3, ..., a_n\}$; Falso, caso contrário.



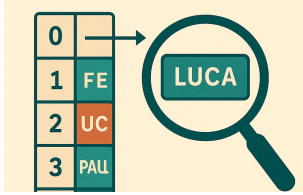
Algoritmos de Busca

- Algoritmos de busca permitem encontrar um dado específico dentro de um conjunto de dados armazenados em estruturas como listas, árvores, matrizes, data frames, etc.
- Retornam se o dado alvo está presente no conjunto de dados, podendo extrair (recuperar) tal dado ou simplesmente retornar sua localização.



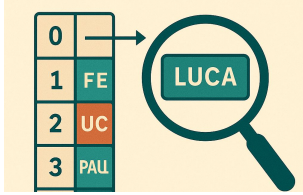
Algoritmos de Busca

- Aplicações em diversas áreas:
  - Localização eficiente de produtos em plataforma de comércio eletrônico.
  - Localizar padrões (pattern matching) em problemas envolvendo reconhecimento (imagem, fala)
  - Localizar ações específicas em problemas de tomada de decisão como ocorre em jogos eletrônicos.
  - Recuperação eficiente de dados em sistemas de base de dados.



Algoritmos de Busca

- Algoritmos abordados:
  - Busca Sequencial
  - Busca Binária



Algoritmo de Busca Sequencial

busca_sequencial(L, chave):

1. Para i= 0 até tamanho(L)-1 faça
2. Se L[i] == chave, então
3. retornar Verdadeiro
4. retornar Falso

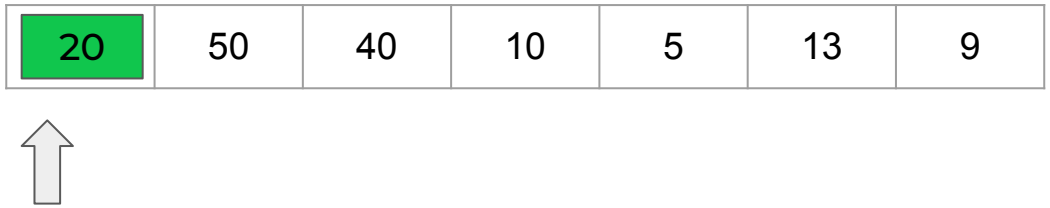
- Verifica sequencialmente cada elemento até localizar ou não o dado procurado.
- A lista não precisa estar ordenada.
- Estratégia simples, fácil de entender e implementar.
- Cada dado da lista pode ser verificado no pior caso.

Algoritmo de Busca Sequencial

busca_sequencial(L, chave):

1. Para i= 0 até tamanho(L)-1 faça
2. Se L[i] == chave, então
3. retornar Verdadeiro
4. retornar Falso

L chave=20

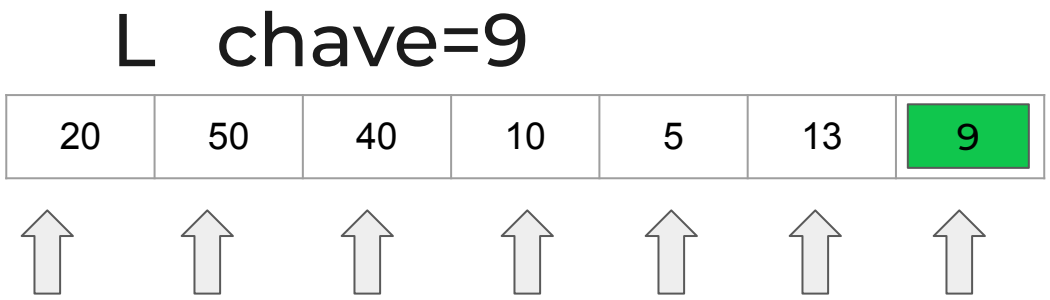


- No melhor caso, o dado procurado está na primeira posição do conjunto de dados.
- Temos complexidade de tempo O(1)
  - Tempo de execução constante

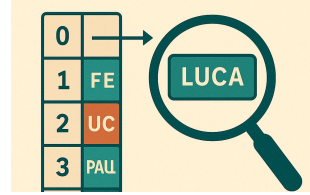
Algoritmo de Busca Sequencial

busca_sequencial(L, chave):

1. Para i= 0 até tamanho(L)-1 faça
2. Se L[i] == chave, então
3. retornar Verdadeiro
4. retornar Falso



- Pior caso: o dado procurado está na última posição ou não está na lista.
- Cada dado da lista é verificado no pior caso.



Algoritmo de Busca Sequencial

```
busca_sequencial(L, chave):
1. Para i= 0 até tamanho(L)-1 faça
2.         Se L[i] == chave, então
3.             retornar Verdadeiro
4. retornar Falso
```

2op: atribuição+soma, len(L) vezes
1op: comparação, len(L) vezes
1op: retorno, 1 vez
1op: retorno, 1 vez

$f(n) = 3n+2$

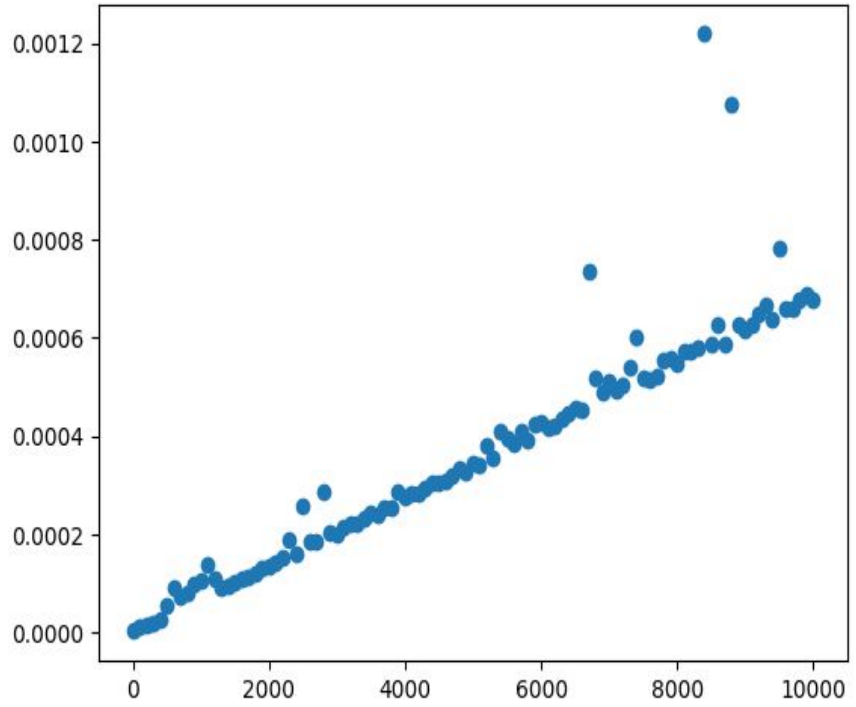
$O(f(n))=O(3n+2)=$**O(n)** ⇒ crescimento de ordem **linear no pior caso**

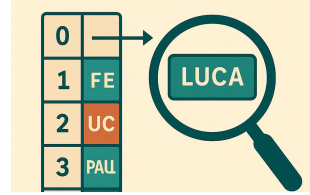
Algoritmo de Busca Sequencial

O(n) ⇒ crescimento de ordem **linear no pior caso**

busca_sequencial(L, chave):

1. Para i= 0 até tamanho(L)-1 faça
2. Se L[i] == chave, então
3. retornar Verdadeiro
4. retornar Falso





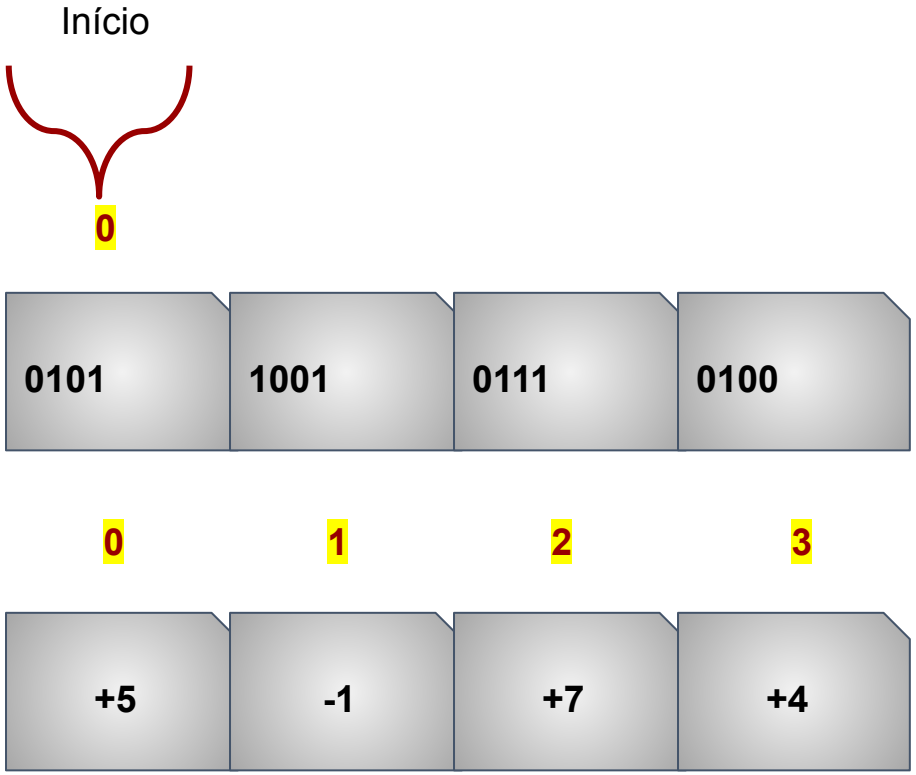
Algoritmo de Busca Sequencial

busca_sequencial(L, chave):

1. Para i= 0 até tamanho(L)-1 faça
2. Se L[i] == chave, então
3. retornar Verdadeiro
4. retornar Falso

- O(len(L)): complexidade linear considerando o tamanho de L, caso cada operação no laço seja executada em tempo constante.
- Python faz isso?
- Python acessa o i-ésimo elemento em tempo constante?

Algoritmo de Busca Sequencial

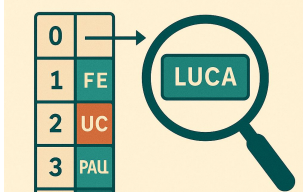


Localização: início +4i

i= 0, início = 0, temos 0 + 4(0) = 0
0101100101110100

i= 1, start = 0, temos 0 + 4(1) = 4
0101100101110100

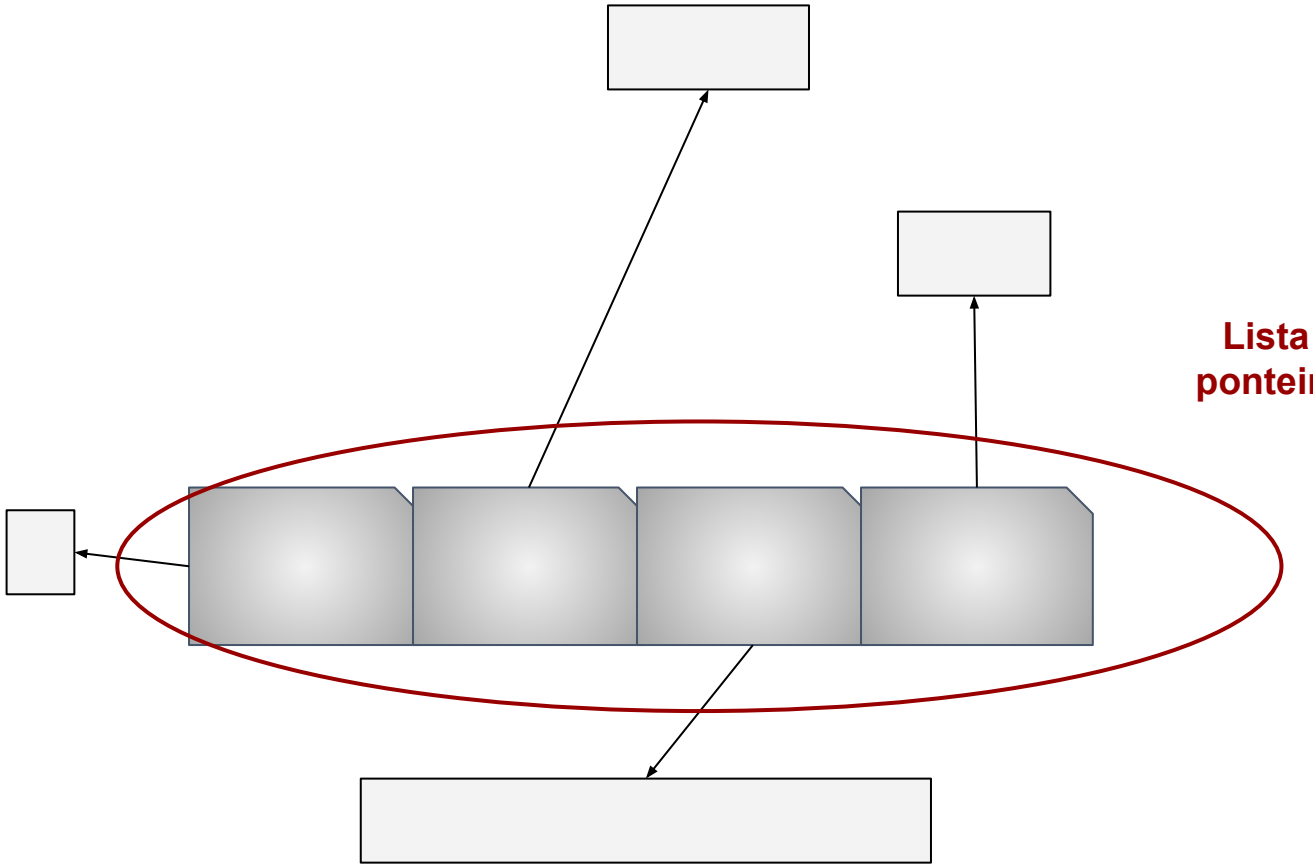
i= 2, start = 0, temos 0 + 4(2) = 8
0101100101110100



Algoritmo de Busca Sequencial

- No exemplo anterior, cada elemento da lista tem o mesmo tamanho.
- O endereço em memória do elemento i da lista é simplesmente início + 4i.
- Python conseguiria calcular tal endereço em tempo constante.

Algoritmo de Busca Sequencial



Lista de ponteiros!!

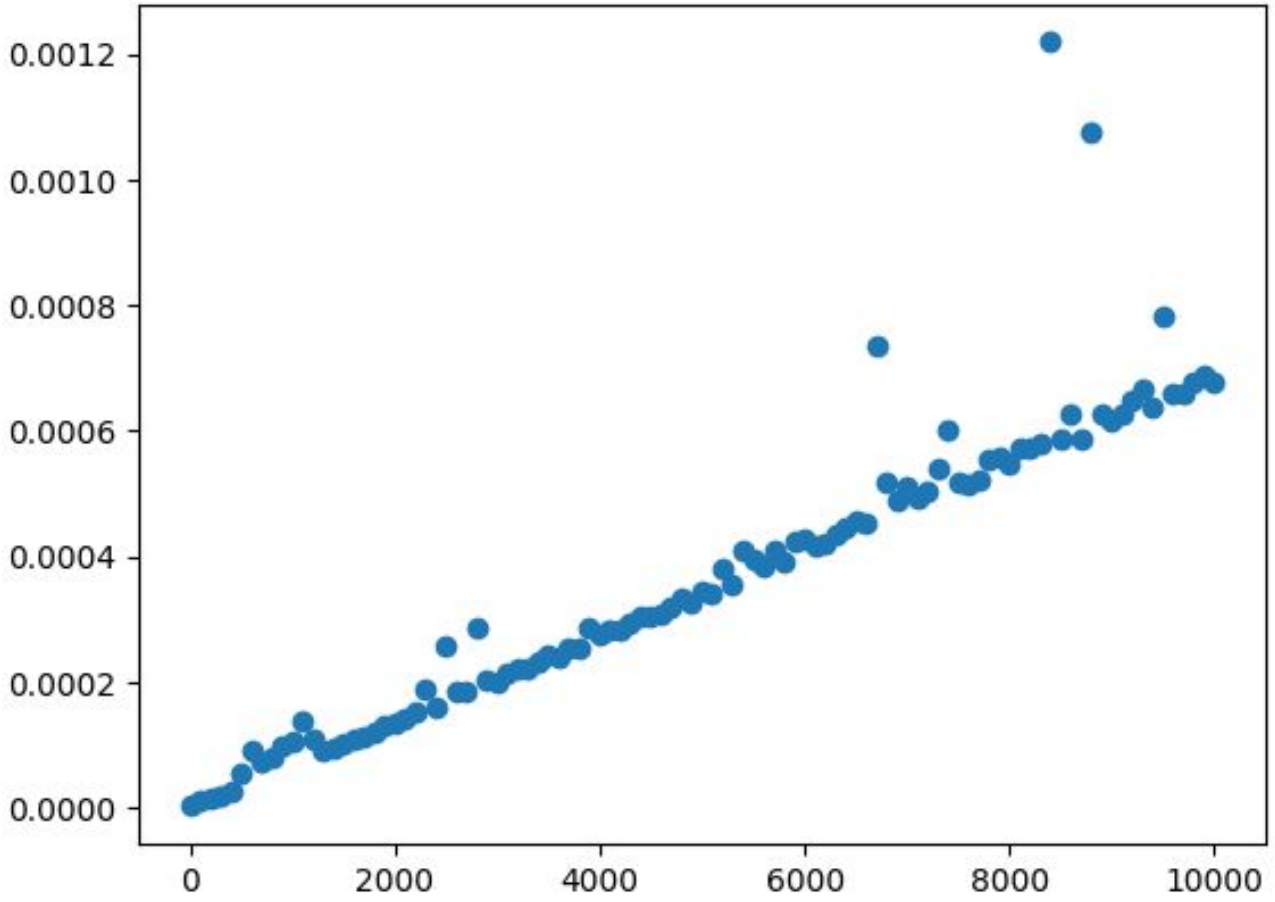
- A lógica anterior não muda, caso consideremos quatro unidades de memória ocupada por ponteiros na lista contendo mais quatro unidades de memória para armazenar endereços dos objetos referenciados.
- O endereço do i-ésimo elemento da lista será armazenado na posição início + 4 + 4i.
- Tal endereço continua sendo determinado em tempo constante.

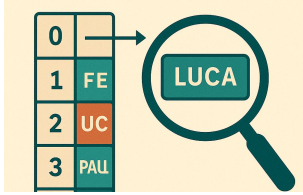
O(n) ⇒ crescimento de ordem

linear no pior caso

busca_sequencial(L, chave):

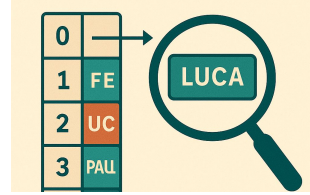
1. Para i= 0 até tamanho(L)-1 faça
2. Se L[i] == chave, então
3. retornar Verdadeiro
4. retornar Falso





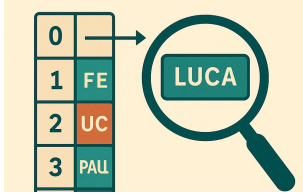
Algoritmos de Busca Binária

- O(n) é o melhor que podemos alcançar em um algoritmo de busca?
  - Sim, se não há uma informação sobre como os elementos se relacionam e sobre a ordem do seu armazenamento na lista.
  - Precisamos verificar cada elemento de L no pior caso!



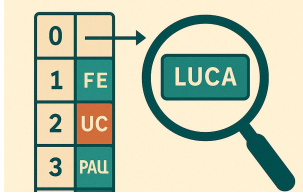
Algoritmos de Busca Binária

- Algoritmos de busca fornecem soluções para o problema computacional de busca.
- Problema Computacional de Busca
  - Input: Uma sequência de n dados $\{a_1\ a_2, a_3, ..., a_n\}$ e dado a_x a ser encontrado.
  - Output: Verdadeiro, se $a_x \in \{a_1\ a_2, a_3, ..., a_n\}$; Falso, caso contrário.



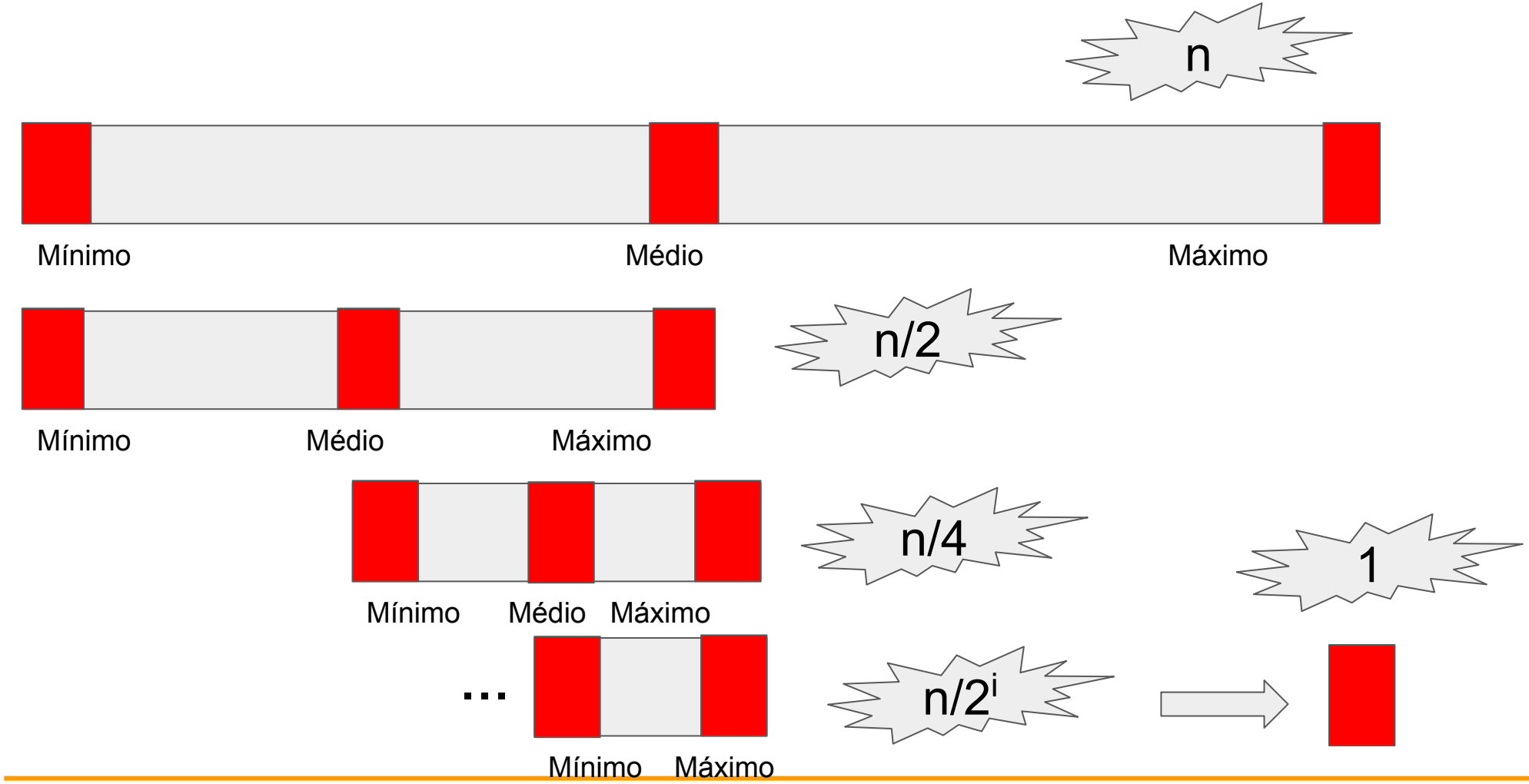
Algoritmos de Busca Binária

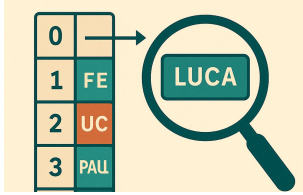
- Agora, vamos mudar o problema de busca:
  - Input: Uma sequência de n dados L={a_1 a_2, a_3, ..., a_n} **ordenados** e dado a_x a ser encontrado.
  - Saída: Verdadeiro, se $a_x \in$ {a_1, a_2, a_3, ..., a_n}; Falso, caso contrário.



Algoritmos de Busca Binária

- Estratégia:
  - Determine um índice para item mediano da lista, ou seja, dividir a lista aproximadamente pela metade.
  - Verifique se L[i] == a_x.
  - Caso contrário, verifique se L[i] é maior ou menor que a_x.
  - A partir da resposta, buscar na metade inferior ou superior da lista.


n
Mínimo
Médio
Máximo
n/2
Mínimo
Médio
Máximo
n/4
Mínimo
Médio
Máximo
1
...
n/2^i
Mínimo
Máximo

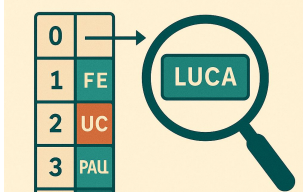


Algoritmos de Busca Binária

buscaBinaria(L, inicio, fim, chave)

1. Se inicio > fim então
   retorna Falso
2. m= (inicio + fim) // 2
3. Se L[m] == chave então
   retorna Verdadeiro
4. Se L[m] > chave então
   retorna buscaBinaria(L, inicio, meio - 1, chave)
5. Senão
   retorna buscaBinaria(L, meio + 1, fim, chave)

- A etapa de decrementar a função, ou seja, reduzir o parâmetro de entrada, tem as seguintes propriedades:
  - Mapeia os valores para os quais os parâmetros ficam limitados a valores não negativos.
  - O valor de decremento é menor em cada chamada recursiva em relação à chamada anterior.

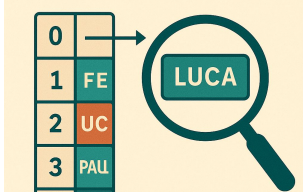


Algoritmos de Busca Binária

buscaBinaria(L, inicio, fim, chave)

1. Se inicio > fim então
   retorna Falso
2. m= (inicio + fim) // 2
3. Se L[m] == chave então
   retorna Verdadeiro
4. Se L[m] > chave então
   retorna buscaBinaria(L, inicio, meio - 1, chave)
5. Senão
   retorna buscaBinaria(L, meio + 1, fim, chave)

- O passo recursivo é encerrado.
  - Quando início > fim.
  - Chave é encontrada

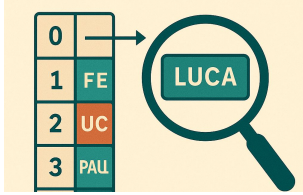


Algoritmos de Busca Binária

buscaBinaria(L, inicio, fim, chave)

1. Se inicio > fim então
    1. retorna Falso
2. m= (inicio + fim) // 2
3. Se L[m] == chave então
    1. retorna Verdadeiro
4. Se L[m] > chave então
    1. retorna buscaBinaria(L, inicio, meio - 1, chave)
5. Senão
    1. retorna buscaBinaria(L, meio + 1, fim, chave)

- buscaBinaria() executa duas chamadas recursivas:
- Uma chamada é ajustada com parâmetros para buscar na metade inferior da lista corrente.

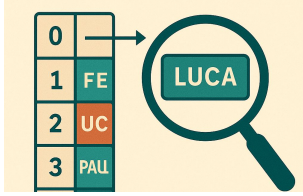


Algoritmos de Busca Binária

buscaBinaria(L, inicio, fim, chave)

1. Se inicio > fim então
    1. retorna Falso
2. m= (inicio + fim) // 2
3. Se L[m] == chave então
    1. retorna Verdadeiro
4. Se L[m] > chave então
    1. retorna buscaBinaria(L, inicio, meio - 1, chave)
5. Senão
    1. retorna buscaBinaria(L, meio + 1, fim, chave)

- buscaBinaria() executa duas chamadas recursivas:
- Outra chamada é ajustada com parâmetros para buscar na metade superior da lista corrente.

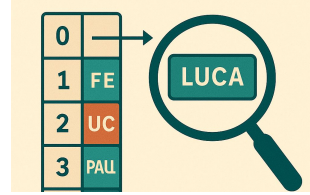


Algoritmos de Busca Binária

buscaBinaria(L, inicio, fim, chave)

1. Se inicio > fim então
    1. retorna Falso
2. m= (inicio + fim) // 2
3. Se L[m] == chave então
    1. retorna Verdadeiro
4. Se L[m] > chave então
    1. retorna buscaBinaria(L, inicio, meio - 1, chave)
5. Senão
    1. retorna buscaBinaria(L, meio + 1, fim, chave)

- buscaBinaria() executa duas chamadas recursivas:
    - Em ambos os casos, a diferença início-fim se reduz à metade.

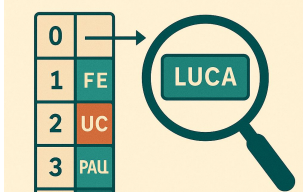


Algoritmos de Busca Binária

buscaBinaria(L, inicio, fim, chave)

1. Se inicio > fim então
    1. retorna Falso
2. m= (inicio + fim) // 2
3. Se L[m] == chave então
    1. retorna Verdadeiro
4. Se L[m] > chave então
    1. retorna buscaBinaria(L, inicio, meio - 1, chave)
5. Senão
    1. retorna buscaBinaria(L, meio + 1, fim, chave)

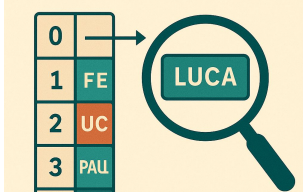
- A implementação assume que L está ordenada.
- Caso L não esteja ordenada, não há obrigação de que o resultado seja aquele esperado.
- Logo, podemos ter:
    - Elemento é localizado
    - Obtemos uma resposta errada.



Algoritmos de Busca Binária

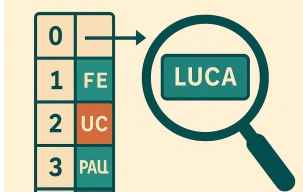
```
buscaBinaria(L, inicio, fim, chave)
    Se inicio > fim então
        retorna Falso
    m= (inicio + fim) // 2
    Se L[m] == chave então
        retorna Verdadeiro
    Se L[m] > chave então
        retorna buscaBinaria(L, inicio, meio - 1, chave)
    Senão
        retorna buscaBinaria(L, meio + 1, fim, chave)
```

- Devemos tratar isso no código?
- Isso ajudaria a eliminar os problemas mencionados...
- ...ao custo O(len(L)) para verificar se a condição é válida, antes de executar a busca.



Algoritmos de Busca Binária

```
busca(L, a_x)
    L = ordena(L)
    chave = a_x
    Se  buscaBinaria(L, 0, tamanho(L)- 1, chave)==Verdadeiro então
        escreve("Elemento encontrado”)
    Senão
        escreve("Elemento não encontrado)
```



Algoritmos de Busca Binária

busca(L, a_x)

L = ordena(L)

chave = a_x

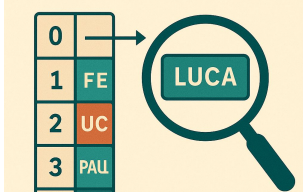
Se buscaBinaria(L, 0, tamanho(L)- 1, chave)==Verdadeiro então

escreve("Elemento encontrado”)

Senão

escreve("Elemento não encontrado)

- busca() é um exemplo de função wrapper:
  - Fornece uma interface para o código do cliente.
  - Trata-se de uma passagem que não faz nenhuma computação séria.
  - Executa a função auxiliar buscaBinaria().



Algoritmos de Busca Binária

busca(L, a_x)

L = ordena(L)

chave = a_x

Se buscaBinaria(L, 0, tamanho(L)- 1, chave)==Verdadeiro então

escreve("Elemento encontrado”)

Senão

escreve("Elemento não encontrado)

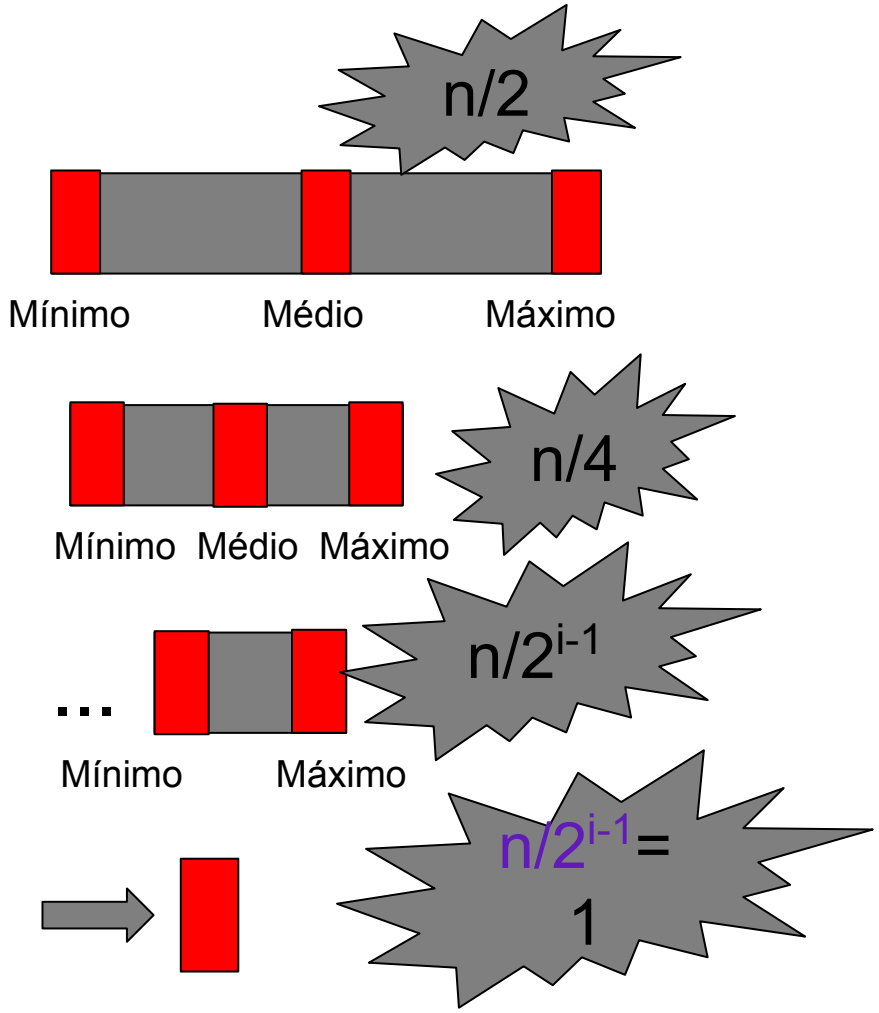
- Podemos eliminar busca() e usar apenas buscaBinaria()!!
- Não!!! Parâmetros como início e fim não tem nada haver com a abstração de procurar um elemento em uma lista.
- Assim como a busca em uma lista já ordenada.
- Tais detalhes de implementação devem ser tratados pelos programas que chamam a busca binária.

Algoritmos de Busca Binária

m= (inicio + fim) // 2

buscaBinaria(L, inicio, meio - 1, chave)

buscaBinaria(L, meio + 1, fim, chave)

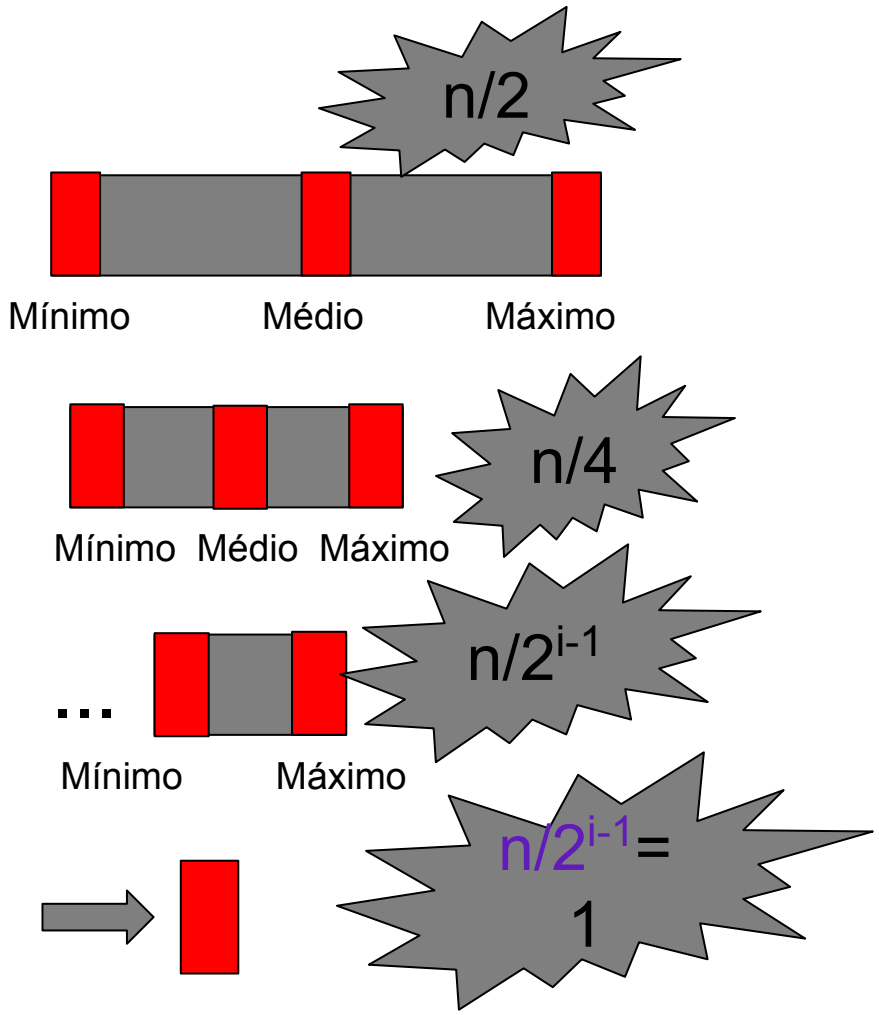


Algoritmos de Busca Binária

- Quando a Busca termina?
  - Seja i o número de passos até chegar ao passo básico da recursão.
  - Temos que $n/2^i=1 \Rightarrow n=2^i$

    $\Rightarrow \log_2 n=\log_2 2^i$

    $\Rightarrow i = \log_2 n$
  - Logo, a complexidade do algoritmo é O(logn).



Algoritmos de Busca Binária

- Lembre-se que $\log_y (x)$ indica o número de vezes que y precisa ser multiplicado para atingir x.
  - $\log_2 8 = x \Leftrightarrow 2^x=8 \Leftrightarrow x=3,\ 2x2x2=8$
- Da mesma forma, dividir 8 por 2 um total de $\log_y (x)$ vezes resulta em 1.
- O termo início–fim é cortado pela metade pelo menos $\log_2$(início–fim) vezes antes de atingir 1.

